

▲ Important Points • • No extra array is needed

• "i" checks every element.

• "j" is the next position for a valid element.

• Skip elements equal to "val"

• At the end, "j" is the number of remaining elements.

▲ Complexity • Brute Force → $O(n)$ & $O(n)$
Main Solution → $O(n)$ & $O(1)$

• Easy, #28, Find the index of the first occurrence in a string.

▲ Problem Statement • Given two strings: "needle" & "haystack",
find the index of the first occurrence of "needle"
in "haystack".

Return "-1" if it is not found.

▲ Main Solution • Try every possible starting position in "haystack"
& compare all characters of "needle".

▲ Brute Force • Check "needle" starting from every index in
"haystack"

▲ Algorithm • 1 • Loop through possible starting position in "haystack"

2 • For each position, compare every character of "needle"

3). If any character doesn't match, move to the next position.

4). If all characters match, return the starting index.

5). If no match is found, return "-1".

▲ Important Points:

- "i" is the starting ~~points~~ position in "haystack".

- "j" checks each character of "needle".

- "haystack [i+j]" compares with "needle [j]"

- Only check positions where the complete "needle" can fit.

- If all characters match, return "i".

▲ Complexity:

Brute Force	→	$O(n \times m)$ & $O(1)$
Main Solution		

Where :- n = length of "haystack"
 m = length of "needle"